

# Constrained MPC-Based Motion Planning for Morphing Quadrotors in Ultra-Narrow Passages under Limited Perception

Harsh Modi<sup>1</sup>, Xiao Liang<sup>2</sup> Minghui Zheng<sup>1</sup>

**Abstract**—This paper introduces a motion planning framework to plan morphology and trajectory for morphing quadrotors under extremely constrained environments. We develop a novel obstacle avoidance cost function for nonlinear model predictive control (MPC) that enables navigation through extremely narrow gaps under limited perception from a 2D LiDAR. Classical artificial potential field-based costs typically have a high cost in narrow passages, artificially blocking the navigable path. In contrast, we propose a smooth exponential obstacle cost that preserves low traversal cost within narrow gaps while maintaining strong collision avoidance behavior. The formulation avoids hard activation thresholds and introduces a cost reduction factor to reduce the cost within narrow passages. Direct use of 2D LiDAR measurements in MPC allows navigation around arbitrarily shaped obstacles. The method is embedded within an acados-based nonlinear MPC framework. Simulation and experimental results demonstrate successful traversal of narrow corridors where typical repulsive cost functions would fail. The approach provides a computationally efficient and practical solution for navigating through tight spaces while maintaining safety from the obstacles. While we are implementing the framework on the morphing quadrotors, the cost function formulation is general-purpose for any mobile robot application, and is not limited to the morphing quadrotors. The implementation code is available at: [Github Repo](#) and a short video is available at: [Video Link](#)

## I. INTRODUCTION

Motion planning for quadrotors has been a widely studied subject since the inception of such robots. However, the motion planning through narrow passages remains challenging. Additionally, the morphing quadrotor designs, such as [1-2], add to the challenge of such planning with varying drone size and dynamics. A variety of methodologies have been proposed to address these challenges.

The model-free approaches utilize reinforcement learning or neural networks to plan and execute the trajectory. The methodology in [3] uses reinforcement learning based coordinated control strategy for morphing quadrotors. However, it lacks explicit dynamic constraints or stability guarantees. The deep reinforcement learning [4] to navigate through tilted narrow gaps using onboard sensing mentions 87.36% success with the Sim2Real method. However, the methodology is applied for fixed frame quadrotors, and the narrow gaps they plan the trajectory through are still larger than the size of

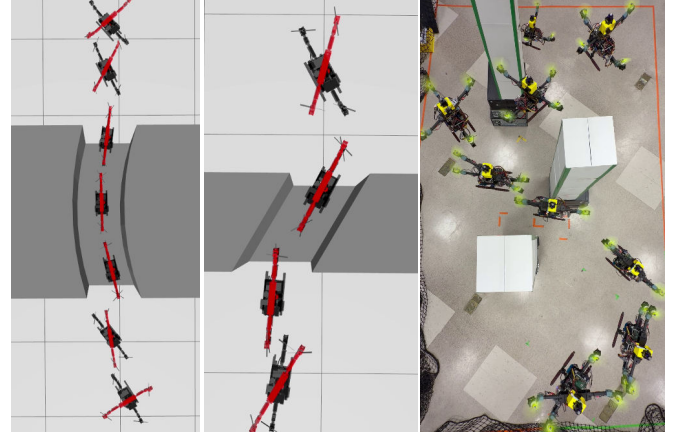


Fig. 1. MorphoCopter autonomously navigating through ultra-narrow passages using proposed MPC-based motion planning in simulations and experiments

the quadrotor. The end-to-end planning and control using a neural network [5] to pass through narrow gaps is also applied to fixed frames, limiting their application to the morphing drone motion planning.

Several methodologies consider some form of dynamic feasibility for planning the trajectories around obstacles for fixed-frame drones. The framework described in [6] uses onboard sensing to plan an aggressive trajectory to pass through tilted narrow gaps. However, it is a local motion planner to just pass through a single narrow gap and requires a large space before and after the narrow gap to execute the planned trajectory. For flying through a cluttered environment, [7] and [8] generate a robust trajectory for fixed frame quadrotors. They first generate kinodynamically feasible B-spline trajectory candidates and then perform collision detection to discard the collision-prone candidates. Method in [9] uses 3D point cloud data to construct a safety corridor between obstacles to plan dynamically feasible trajectories. However, it ensures dynamic feasibility by limiting velocity and acceleration up to a maximum limit and does not consider explicit model-based feasibility. The method in [10] plans a trajectory through narrow gaps under disturbances for fixed-frame drones.

For morphing drones, there are some studies to plan the trajectories among narrow gaps in a cluttered environment. The planner in [11] is a B-spline-based trajectory planner for morphing quadrotors. It considers a point cloud-based safe corridor construction, but does not consider changing moments of inertia for planning the optimum trajectory through obstacles. [12] is a trajectory planner and controller

This work was partially supported by U.S. National Science Foundation (Grants: No. 2422698). Correspondence to Minghui Zheng.

<sup>1</sup>Harsh Modi harsh.modi@tamu.edu and Minghui Zheng mhzheng@tamu.edu are with the Department of Mechanical Engineering, Texas A&M University, College Station, TX 77843, USA.

<sup>2</sup>Xiao Liang xliang@tamu.edu is with the Department of Civil & Environmental Engineering, Texas A&M University, College Station, TX 77843, USA.

that plans a morphing strategy using a pre-constructed point cloud map of the environment. But it cannot replan the trajectory for unknown/changed obstacles. [13] is a low-level attitude controller for the morphing quadrotors, but does not incorporate trajectory planning or obstacle avoidance.

On the other hand, Model Predictive Control (MPC) has been widely used for controlling dynamic systems, particularly where actuator and environmental constraints must be rigorously respected. As an optimization-based control framework, MPC operates by predicting future system behavior over a receding horizon and solving constrained optimization problems online. This enables it to generate optimal control inputs that minimize tracking error and control effort while ensuring constraint satisfaction. Its ability to incorporate system models and handle nonlinearities makes it an attractive choice for morphing quadrotors, which inherently involve changing dynamics and strong coupling between structure and control.

[14] is an example of planning the trajectory using MPC for a fixed-frame quadrotor. It introduces a trajectory generation through moving obstacles using chance constraints (i.e. by restricting collision probability). It considers a constraint of minimum safety distance from the nearest obstacle in the MPC. For morphing drones, [15], [16], and [17] utilize various MPC-based approaches to plan the trajectories and are very close to our method presented here. [15] uses the minimum snap trajectory planning with augmented MPC and considers moments of inertia of the morphing quadrotor. However, it assumes a static environment and does the planning before the flight. [16] uses a modified A\* algorithm to plan and generate a trajectory through extremely narrow gaps, along with nonlinear MPC to ensure the dynamical feasibility. However, this methodology also assumes a static environment and does not have online replanning capability. [17] is a similar approach with high level planner and nonlinear model predictive control for reconfigurable drones passing through narrow gaps. However, it considers only cubic or cylindrical well-defined narrow gaps, and also cannot plan through unknown obstacles.

The obstacle avoidance cost functions introduced in prior methodologies loosely resemble the Artificial Potential Field (APF) first introduced by Khatib [18]. The APF utilizes the concept of repulsive force from the obstacles. However, the classic APF-based cost functions fail to plan a trajectory through extremely narrow gaps [19-20] as the space within the narrow gap is extremely close to the obstacles. This results in motion planners being unable to find a solution through narrow gaps, even if feasible collision-free paths exist. The other constraint-based approaches typically use control barrier functions or signed-distance constraints to guarantee safety [21-22]. These methods provide strong safety guarantees, but can introduce non-convex constraints and increase computational complexity.

To overcome the issues mentioned, our methodology uses the following:

- 1) We introduce an exponential baseline obstacle avoidance cost function, which has a smooth gradient for

optimizers without any hard distance cutoff.

- 2) The cost function incorporates a novel cost reduction factor, which results in a low cost within extremely narrow gaps, enabling traversal where traditional repulsive potentials would create artificial barriers.
- 3) Operate using limited 2D LiDAR perception, which can plan the trajectory around arbitrarily shaped obstacles.
- 4) The methodology has been applied to a morphing quadrotor (MorphoCopter) to autonomously navigate through cluttered environment with automatic folding and unfolding.

By embedding this cost formulation within an acados-based [23] nonlinear MPC framework, the proposed approach combines real-time dynamic feasibility with gap-aware obstacle cost shaping, enabling robust navigation through arbitrarily shaped obstacles under limited sensing. The implementation is done on the morphing quadrotor platform MorphoCopter [2].

The rest of the article is organized as follows: Section II introduces the overall implementation framework. Section III introduces the MPC formulation and novel cost function proposed in this article. Section IV showcases the simulation and experiment results with the applied methodology and compares with a baseline cost function. Section V concludes the article.

## II. IMPLEMENTATION FRAMEWORK

The implementation framework used is summarized in Fig. 2. We combine a high-level planner, a trajectory planner, and a low-level trajectory follower to develop the holistic framework. The high-level planner plans a rough path around previously known static obstacles, the trajectory planner plans a precise trajectory online around any present obstacles, and the low-level trajectory follower running at high frequency ensures that the MorphoCopter is following the planned trajectory.

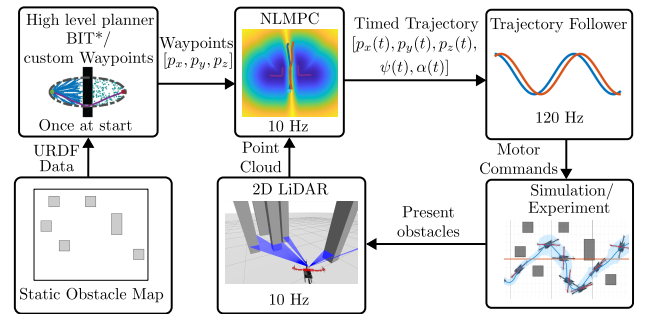


Fig. 2. Implementation framework, partial image source: [24]

The purpose of the high-level planner is to plan a reference trajectory around the previously known static obstacles. We are using Batch Informed Trees (BIT\*) [24] algorithm for high-level path planning due to its higher convergence speed, informed sampling, and being an anytime algorithm. The BIT\* is implemented using the Open Motion Planning Library (OMPL). The search is performed for position  $x$

( $p_x$ ), position y ( $p_y$ ), position z ( $p_z$ ), and yaw in the BIT\*. The joint angle is kept constant at  $\pi/2$  rad corresponding to the minimum width of the MorphoCopter. This ensures that the BIT\* can find the solution even in the extremely narrow gaps. After the solution is found, the 3D waypoints are stored in the CSV file to be utilized by the online trajectory planner discussed next. It is important to note that we do not need to use a specific high-level planner; the online MPC planner (main contribution of this work) just needs a reference trajectory to follow a general direction. In some of our simulations and experiments, we just provide the straight line reference to demonstrate the ability of the online planner.

We use Model Predictive Control (MPC) as the trajectory planner with the following advantages:

- MPC can consider varying dynamics of the MorphoCopter explicitly and only plan a feasible trajectory at a given configuration.
- We can provide a custom cost function to minimize the trajectory errors and control costs while avoiding obstacles.
- We can provide a reference folding angle such that it prefers an unfolded, more efficient configuration, and only folds into a compact configuration if required to avoid obstacles.

For a low-level trajectory follower, the PID controller as described in [2] is used. The trajectory follower runs at a much higher rate (120 Hz) compared to the MPC (10 Hz). The low-level controller ensures that the MorphoCopter remains stable even in the event of a delay in calculation from the MPC-based trajectory planner.

In the next section, we will describe the MPC formulation and introduce the novel obstacle avoidance cost function.

### III. MPC WITH NOVEL COST FUNCTION

The necessary components for the MPC are system dynamics, cost function or objective function, and reference trajectory. We will discuss each of them for our MPC formulation in the upcoming subsections.

#### A. System Dynamics

The MPC for MorphoCopter has 14 states as described in table I. Compared to a standard quadrotor MPC, we have 2 extra states corresponding to the joint angle. Out of these, we consider  $p_x$ ,  $p_y$ ,  $p_z$ ,  $\psi$ , and  $\alpha$  as outputs of the MPC. The inputs are 4 motor thrusts ( $u_1, u_2, u_3, u_4$ ) and the joint angle torque  $u_\alpha$ .

TABLE I  
MPC STATES

| State                    | Symbol                 | Description                        |
|--------------------------|------------------------|------------------------------------|
| $\mathbf{x}(1, 2, 3)$    | $p_x, p_y, p_z$        | Positions in $x, y, z$             |
| $\mathbf{x}(4, 5, 6)$    | $v_x, v_y, v_z$        | Velocities in $x, y, z$            |
| $\mathbf{x}(7, 8, 9)$    | $\phi, \theta, \psi$   | Roll, pitch and yaw angles         |
| $\mathbf{x}(10, 11, 12)$ | $p, q, r$              | Angular rates about body $x, y, z$ |
| $\mathbf{x}(13, 14)$     | $\alpha, \dot{\alpha}$ | Joint angle and its rate of change |

These states evolve with time as shown in equations below:

$$\begin{aligned}\dot{\mathbf{x}}(1) &= \dot{p}_x = v_x = \mathbf{x}(4) \\ \dot{\mathbf{x}}(2) &= \dot{p}_y = v_y = \mathbf{x}(5) \\ \dot{\mathbf{x}}(3) &= \dot{p}_z = v_z = \mathbf{x}(6)\end{aligned}\quad (1)$$

$$\begin{bmatrix} \dot{\mathbf{x}}(4) \\ \dot{\mathbf{x}}(5) \\ \dot{\mathbf{x}}(6) \end{bmatrix} = \begin{bmatrix} \ddot{p}_x \\ \ddot{p}_y \\ \ddot{p}_z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \frac{\mathbf{R}}{m} \begin{bmatrix} 0 \\ 0 \\ -T \end{bmatrix} \quad (2)$$

where  $g$  is the gravitational constant,  $\mathbf{R}$  is the rotation matrix from world frame to MorphoCopter body frame, and  $m$  is the MorphoCopter mass. The total thrust  $T$  in the body  $z$  axis is given by:

$$T = (u_1 + u_2 + u_3 + u_4) \cdot \cos(\delta) \quad (3)$$

where  $\delta$  is the fixed tilt of the motor as described in [2].

$$\begin{bmatrix} \dot{\mathbf{x}}(7) \\ \dot{\mathbf{x}}(8) \\ \dot{\mathbf{x}}(9) \end{bmatrix} = \begin{bmatrix} \dot{\phi} \\ \dot{\psi} \\ \dot{\theta} \end{bmatrix} = \mathbf{N}^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (4)$$

where  $\mathbf{N}$  is the matrix relating Euler angles with body angular rates given by:

$$\mathbf{N} = \begin{bmatrix} 1 & 0 & \sin(\theta) \\ 0 & \cos(\phi) & \cos(\theta) \sin(\phi) \\ 0 & -\sin(\phi) & \cos(\theta) \cos(\phi) \end{bmatrix} \quad (5)$$

$$\begin{bmatrix} \dot{\mathbf{x}}(10) \\ \dot{\mathbf{x}}(11) \\ \dot{\mathbf{x}}(12) \end{bmatrix} = \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \mathbf{I}^{-1} \left( \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} - \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \mathbf{I} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \right) \quad (6)$$

where,  $\mathbf{I}$  is the inertia matrix,  $\tau_x, \tau_y, \tau_z$  are body moment generated around body  $x, y, z$  axes respectively.

$$\begin{aligned}\dot{\mathbf{x}}(13) &= \dot{\alpha} = \mathbf{x}(14) \\ \dot{\mathbf{x}}(14) &= \ddot{\alpha} = \frac{u_\alpha}{\mathbf{I}(3, 3)}\end{aligned} \quad (7)$$

#### B. Cost Function

The cost function at each stage (timestep) is constructed using a combination of trajectory deviation cost, control cost, and obstacle avoidance cost as:

$$J_j = J_{y,j} + J_{u,j} + J_{o,j} \quad (8)$$

where  $J_{y,j}$  is the cost penalizing the trajectory deviation from the reference trajectory,  $J_{u,j}$  penalizes the control inputs that deviate from the hover controls, and  $J_{o,j}$  is the obstacle avoidance cost.  $J_{y,j}$  and  $J_{u,j}$  are standard nonlinear least square cost terms.  $J_{y,j}$  is given by:

$$J_{y,j} = \frac{1}{2} (\mathbf{y}_j - \mathbf{y}_j^{\text{ref}})^\top \mathbf{W}_{y,j} (\mathbf{y}_j - \mathbf{y}_j^{\text{ref}}) \quad (9)$$

Here  $\mathbf{W}_{y,j} \in \mathbb{R}^{n_y \times n_y}$  is a tunable trajectory deviation cost weight matrix.  $\mathbf{y}_j$  is the output state of the MPC at stage  $j$ , and  $\mathbf{y}_j^{\text{ref}}$  is the reference output state.  $J_{u,j}$  is given by:

$$J_{u,j} = \frac{1}{2} (\mathbf{u}_j - \mathbf{u}_j^{\text{ref}})^\top \mathbf{W}_{u,j} (\mathbf{u}_j - \mathbf{u}_j^{\text{ref}}) \quad (10)$$

Here  $\mathbf{W}_{u,j} \in \mathbb{R}^{n_u \times n_u}$  is a control cost weight matrix.  $\mathbf{u}_j$  is the control input at stage  $j$  and  $\mathbf{u}_j^{\text{ref}}$  is the reference control input, which corresponds to the hover controls in our case.

The obstacle avoidance cost function proposed in this article,  $J_{o,j}$ , is given by:

$$J_o = W_o \cdot (1 - (\mu^2 - 1)^2) \cdot \exp\left(1 - \frac{d^{*2}}{d_0^2}\right) \quad (11)$$

Here  $W_o \in \mathbb{R}$  is the obstacle avoidance weight parameter.  $d^*$  is the distance to the closest detected obstacle from the prediction location.  $d_0$  is the distance where the cost is the same as the weight  $W_o$ . The cost increases as  $d^*$  approaches 0 and decreases as it approaches infinity. The cost term does not have any sharp cutoff, resulting in a smoother gradient in the whole region, unlike standard APF-like cost functions.  $\mu$  is the novel cost reduction factor responsible for reducing the cost in narrow passages, which eliminates the problem faced in the standard APF-like cost functions. We will discuss the cost reduction factor in the next subsection.

### C. Cost Reduction Factor

The cost reduction factor ( $\mu$ ) reduces the obstacle cost in the narrow passages, even if the points inside the passage are very close to the obstacles. This is done to ensure that the MPC can generate trajectories through extremely narrow gaps. The calculation of the cost reduction factor is explained using Algorithm 1, Algorithm 2, and Fig. 3 with an example. The detailed methodology is described below.

---

**Algorithm 1** Line segment extraction from clustered point cloud

---

**Require:** Clustered 3D points  $\mathcal{D}$ , clustering threshold  $\varepsilon$

**Ensure:** Set of merged line segments  $\mathcal{L}$

- 1: **for** each cluster  $C_i \in \mathcal{D}$  **do**
  - 2:   Sort points in  $C_i$  by original index
  - 3:   Detect break points  $\mathcal{B}$  based on distance trends and discontinuities
  - 4:   Split  $C_i$  into sub-clusters  $\{S_1, S_2, \dots, S_m\}$  at  $\mathcal{B}$
  - 5:   **for** each sub-cluster  $S_k$  **do**
  - 6:     Fit a 3D line via linear regression
  - 7:     Store its endpoints as a line segment  $L_k$
  - 8:   **end for**
  - 9:   Merge adjacent line segments if nearly parallel and spatially close
  - 10: **end for**
  - 11: **return**  $\mathcal{L}$
- 

In the pre-processing, the 2D LiDAR point cloud is processed to generate line segments corresponding to the detected obstacles using Algorithm 1. First, the LiDAR point cloud data is clustered using the DBSCAN algorithm. The clustering threshold ( $\varepsilon$ ) can be tuned as per the size of the robot, so that the gaps smaller than the size of the robot do not result in different clusters. These clusters are then further divided if the angle between the consecutive points is more than the threshold (i.e.  $20^\circ$ ). Using this threshold, we can approximate curved shapes using line segments. Then, for each cluster, line segments are fitted using linear regression. These line segments represent the obstacle boundaries facing the LiDAR sensor.

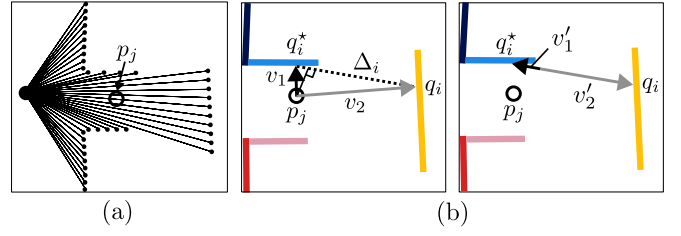


Fig. 3. (a) 2D LiDAR scan (b) Cost reduction factor calculation (see Algorithm 2)

The extracted line segments are provided to the acados MPC loop to calculate the cost reduction factor using Algorithm 2, which can be graphically visualized in Fig. 3. From each MPC prediction position ( $p_j$ ), we determine the closest point on each obstacle line segment ( $q_i$ ) and calculate the vectors ( $v_i$ ) connecting  $p_j$  to the closest points on each line segment. Out of all these vectors, we make the pairs of the smallest vector ( $v_1 = v_i^*$ ) with every other vector  $v_i$ . Now, for each such pair, we project the vectors onto the line ( $\Delta_i$ ) connecting the respective points on the line segments. Then, we take the normalized vector sum of these projections ( $s_i$ ). The final cost reduction factor for timestep  $j$  is the minimum of all such normalized vector sums from all the line segments.

---

**Algorithm 2** Narrow gap cost reduction factor calculation

---

**Require:** Line segments  $\mathcal{L}$ , 2D part of MPC predicted positions  $p_j$

**Ensure:** Cost reduction factor  $\mu_j$  for each MPC prediction timestep  $j$

- 1: **for** each timestep  $j$  **do**
  - 2:   Let  $p_j$  be the predicted position at timestep  $j$
  - 3:   {Step 1: Compute closest obstacle points}
  - 4:   **for** each segment  $i$  in  $\mathcal{L}$  **do**
  - 5:     Compute closest point  $q_i$  on segment  $i$  to  $p_j$
  - 6:     Compute distance between  $p_j$  and  $q_i$
  - 7:   **end for**
  - 8:   Select index  $i^*$  with minimum distance
  - 9:   Let  $q^* \leftarrow q_{i^*}$
  - 10:   {Step 2: Evaluate narrow-gap interaction}
  - 11:   Initialize  $\mu_j \leftarrow 1$
  - 12:   **for** each segment  $i$  in  $\mathcal{L}$  **do**
  - 13:     Compute closest point  $q_i$  to  $p_j$
  - 14:     Compute passage direction  $\Delta_i \leftarrow q^* - q_i$
  - 15:     Compute vectors from predicted position to obstacle points:  $v_1 \leftarrow q^* - p_j$ ,  $v_2 \leftarrow q_i - p_j$
  - 16:     Project  $v_1$  and  $v_2$  onto  $\Delta_i$  as  $v'_1$  and  $v'_2$
  - 17:     Compute passage scaling using normalized sum of projections  $s_i \leftarrow \frac{\|v'_1 + v'_2\|}{\|v'_1\| + \|v'_2\|}$
  - 18:     Update  $\mu_j \leftarrow \min(\mu_j, s_i)$
  - 19:   **end for**
  - 20: **end for**
  - 21: **return**  $\{\mu_j\}$
- 

With this algorithm, the cost reduction factor for a given location is 0 if it is exactly in the middle of the narrow



passage and is 1 if a given point has an obstacle only on one side (i.e., it's not a narrow passage).  $\mu = 0$  will result in the obstacle avoidance cost of 0 as per (11), allowing the MPC to plan the trajectory inside a narrow passage.  $\mu = 1$  will not affect the obstacle avoidance cost, resulting in the obstacle cost same as the base cost.

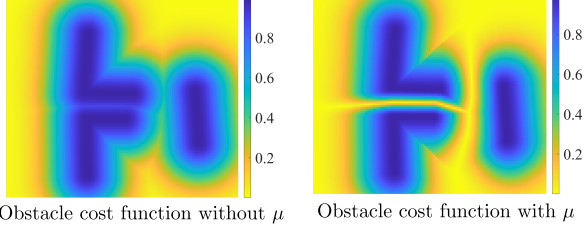


Fig. 4. Comparison of the obstacle avoidance cost function without and with cost reduction factor

Fig. 4 shows the effect of the cost reduction factor on the obstacle cost. As shown in this comparison, the obstacle avoidance cost is very high (close to maximum) inside the narrow gap if the cost reduction factor is not used. This is due to the distance between the points inside the narrow passage and the obstacles being very small. But with the cost reduction factor, the cost function has a passage corresponding to the opening among the obstacles. If the cost reduction factor is not considered, the MPC will not be able to find paths through the narrow gap.

#### D. Reference Trajectory

As previously mentioned, the reference trajectory is provided by the BIT\* algorithm or by custom waypoints. We only use 3D location data from the reference trajectory and directly feed it into  $\mathbf{y}_j^{\text{ref}}$  in (9). For yaw reference and joint angle reference, we utilize the MPC trajectory generated in the previous MPC iteration:

##### 1) Yaw Reference:

$$\psi_j^{\text{ref}} = \tan^{-1} \left( \frac{p_{y_{j+1}} - p_{y_j}}{p_{x_{j+1}} - p_{x_j}} \right) \text{ for } 1 < j < n - 1 \quad (12)$$

and for  $j = n$ , we consider  $\psi_n^{\text{ref}} = \psi_{n-1}^{\text{ref}}$ . Here  $n$  is the number of timesteps used in MPC.

2) Joint Angle Reference: For determining the joint angle reference, we first calculate the maximum half-width of the MorphoCopter using:

$$w = \max(d_{b_y}^* - d_{\min}, p_l/2) \quad (13)$$

Here,  $p_l$  is the propeller diameter,  $d_{b_y}^*$  is the distance to the closest obstacle projected along the body  $y$  axis, and  $d_{\min}$  is the safety distance desired between the MorphoCopter and the nearest obstacle.

Now, if  $w > l \cdot \cos(\pi/4) + p_l/2$ , then the reference location does not need any folding and hence  $\alpha_j^{\text{ref}} = 0$ . Here,  $l$  is the distance from the center of mass of the MorphoCopter to the propeller center. If  $w \leq l \cdot \cos(\pi/4) + p_l/2$ , we can calculate the reference joint angle using:

$$\alpha_j^{\text{ref}} = \frac{\pi}{2} - 2 \cdot \sin^{-1} \left( \frac{w - p_l/2}{l} \right) \quad (14)$$

The 3D location reference received from BIT\*,  $\psi_j^{\text{ref}}$ , and  $\alpha_j^{\text{ref}}$  combined make the the reference trajectory  $\mathbf{y}_j^{\text{ref}}$ , which is entered into (9).

#### E. Longitudinal and Lateral Cost Weights

In order to allow the MPC trajectory to deviate from the reference trajectory for avoiding the obstacles, while still following the reference trajectory's general direction, we use different cost weights in the longitudinal and lateral directions. The lateral direction cost is generally desired to be much lower than the longitudinal direction cost. As the MPC requires standard x-y cost, we convert the longitudinal-lateral cost weights into x-y cost weights for each timestep using the following:

$$\mathbf{W}_{y,j}^{p_x, p_y} = \begin{bmatrix} w_v \cdot c^2(\psi_j) + w_h \cdot s^2(\psi_j) & (w_v - w_h) \cdot s(\psi_j)c(\psi_j) \\ (w_v - w_h) \cdot s(\psi_j)c(\psi_j) & w_v \cdot s^2(\psi_j) + w_h \cdot c^2(\psi_j) \end{bmatrix} \quad (15)$$

where  $w_v$  is the tunable parameter for the longitudinal cost and  $w_h$  is the tunable parameter for the lateral cost.  $c$  represents  $\cos$  and  $s$  represents  $\sin$ .

#### F. Constraints

One of the advantages of the MPC is the explicit use of the constraints. We use the following constraints in our formulation:

- 1)  $T_{\text{idle}} \leq u_i \leq T_{\text{max}}$  for  $i = 1, 2, 3, 4$ ; where  $T_{\text{idle}}$  is the thrust generated by motors while idling and  $T_{\text{max}}$  is the physical limit of the thrust the specific motors can produce.
- 2)  $u_\alpha \leq \tau_{\alpha, \text{max}}$ ; where  $\tau_{\alpha, \text{max}}$  is the torque limit of the joint angle servo motor used.
- 3)  $0 \leq \alpha \leq \pi/2$ ; where  $\alpha$  is the joint angle.

These constraints ensure that the planned trajectory respects the control limits and remains within the hardware structural limit. Next, we will discuss the results from the simulations and experiments conducted using the framework described here.

## IV. RESULTS

We implement the framework with the proposed obstacle avoidance cost function in various challenging scenarios in simulations and experiments to validate the performance and to compare with standard APF-like cost functions. This includes navigating through various narrow passages and previously unknown obstacles (unknown to BIT\*). The tunable parameters used in our specific case are described in Table II. We will delve into the details of the simulations and experiments in the subsections below. For better visualization, please refer to the accompanying video.

#### A. Simulations

The simulations are performed in high fidelity simulator Gazebo using PX4 firmware. Virtual LiDAR sensor is added on top of the MorphoCopter to detect the obstacles in the environment. For some cases, straight path references are

TABLE II  
TUNABLE PARAMETERS USED

| Description     | Parameters                                 | Values                                        |
|-----------------|--------------------------------------------|-----------------------------------------------|
| MPC Runtime     | $n, t_{del}$                               | 25, 0.1 s                                     |
| Trajectory cost | $w_v, w_h, \mathbf{W}_y^{z, \psi, \alpha}$ | 0.53, 0.026, $diag(4.0, 2.0, 20.0)$           |
| Control cost    | $\mathbf{W}_u$                             | $diag(0.9, 0.9, 0.9, 0.9, 9.5 \cdot 10^{-5})$ |
| Obstacle cost   | $d_{min}, d_0, W_o$                        | 0.16 m, 0.6 m, 1.0                            |

considered instead of using BIT\* to make the scenarios more challenging for MPC.

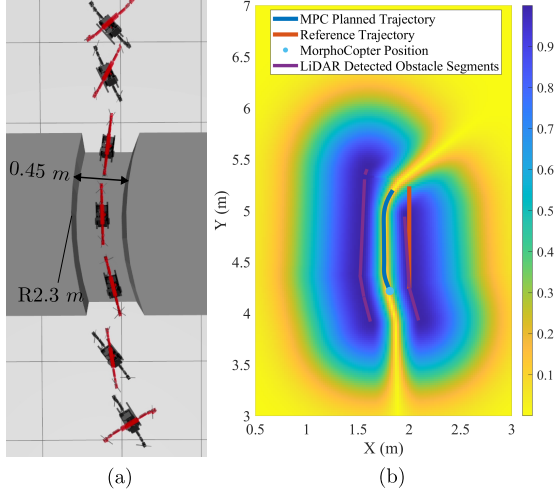


Fig. 5. (a) MorphoCopter navigating using MPC through curved narrow passage in simulation (b) Obstacle avoidance cost function for one of the iterations

1) *Curved and inclined narrow passages:* In the first simulation scenario, extremely narrow gaps (less than the size of the MorphoCopter) of curved and inclined surfaces are generated as shown in Fig. 5 (a) and Fig. 6 (a). The curved narrow passage is of 0.45 m width and has a radius of 2.3 m. The width of the narrow gap is narrower than the size of the MorphoCopter in the unfolded configuration, hence it has to morph to a narrower configuration to pass through the narrow gap. Fig. 5 (a) shows the sequence of the MorphoCopter navigating through the narrow gap, and Fig. 5 (b) shows the obstacle avoidance cost function for one of the iterations. A narrow path corresponding to the middle of the gap is generated in the cost map using the cost reduction factor introduced in this article. Even with limited 2D LiDAR sensing, the cost function is adequate to navigate through this challenging environment. Please note that all the cost maps shown in this article are generated for illustrative purposes. The MPC does not need to compute the whole cost map, but calculates the cost just for the predicted positions.

Similarly, Fig. 6 (a) shows the sequence of the MorphoCopter passing through 0.4 m horizontally inclined passage, and Fig. 6 (b) shows the cost function for one of the iterations. The reference trajectory was deliberately chosen to be offset from the narrow passage and of a straight path to showcase the ability of the MPC framework to find the safe path through the corridor.

2) *Cluttered obstacles:* In this simulation scenario, a cluttered obstacle environment with multiple obstacles placed

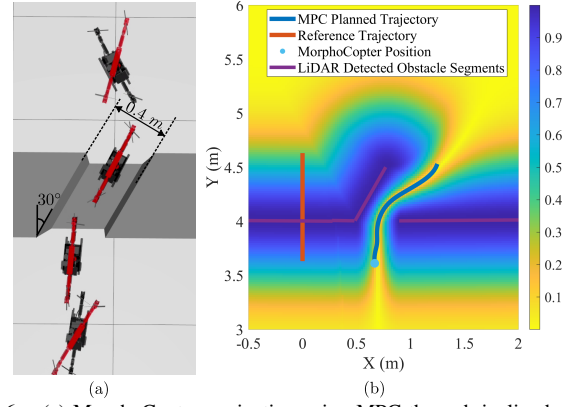


Fig. 6. (a) MorphoCopter navigating using MPC through inclined narrow passage in simulation (b) Obstacle avoidance cost function for one of the iterations

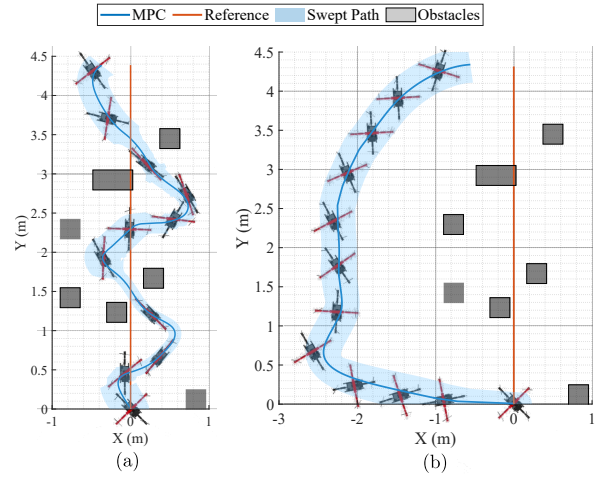


Fig. 7. MorphoCopter navigating through a cluttered environment in simulations using (a) proposed cost function (b) standard repulsive cost function

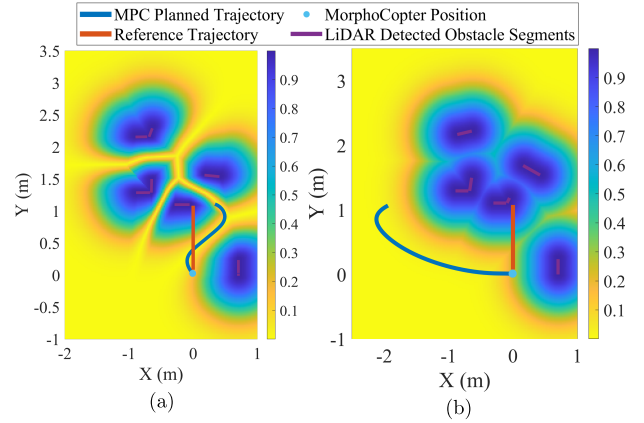


Fig. 8. (a) Proposed cost function with cost reduction factor (b) Standard repulsive cost function

very close to each other was generated. There are various narrow passages among the obstacles that are narrower than the width of the MorphoCopter in the unfolded configuration. In this simulation also, we provide a straight line reference trajectory without using BIT\* to demonstrate the ability of the MPC. Fig. 7 (a) shows the MorphoCopter navigating through this environment. The figure also shows the swept path by the MorphoCopter, which covers the envelope of the size of the MorphoCopter, considering the folding angle.

We can see that with the proposed framework, it is able to navigate through this environment comfortably while folding as needed. For benchmark purposes, the simulation without a cost reduction factor was also performed as shown in Fig. 7 (b). Without the cost reduction factor, MPC cannot find a path between obstacles and has to plan around to avoid them. This results in the average deviation of  $2.6\text{ m}$  from the reference trajectory, which is only  $0.41\text{ m}$  in the case where we use the cost reduction factor.

The effect of the cost reduction factor can be more easily visualized in Fig. 8. Fig. 8 (a) shows the initial cost function with cost reduction factor, while Fig. 8 (b) shows the cost function without cost reduction factor. The figure clearly illustrates that there is no opening in the “no cost reduction factor” case, forcing the MPC trajectory around the obstacles.

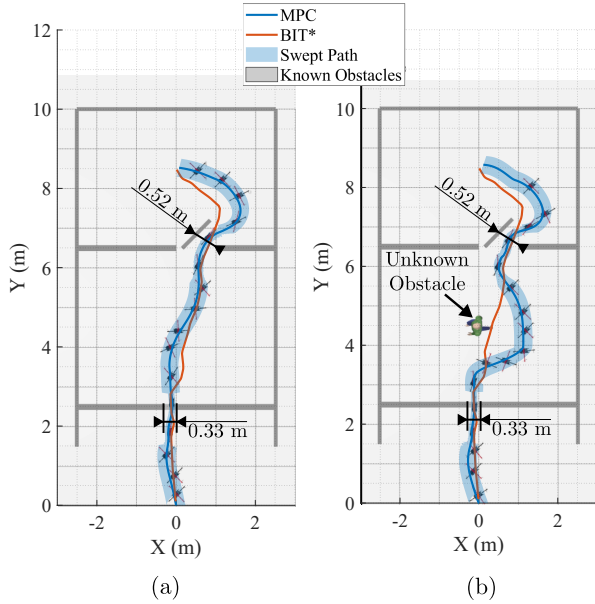


Fig. 9. MorphoCopter navigating through a realistic room environment in simulations (a) without an unknown obstacle (b) with an unknown obstacle

3) *Realistic room*: In this simulation scenario, a realistic room with a window and an ajar door was constructed as shown in Fig. 9. The window creates a narrow gap with  $0.33\text{ m}$  width, and the ajar door creates an inclined narrow passage of  $0.52\text{ m}$  width. In this simulation, we utilize BIT\* to generate the reference trajectory. Fig. 9 (a) shows the environment obstacles, BIT\* reference trajectory, and the MPC trajectory along with the swept path considering the size of the MorphoCopter and the snapshots from the simulation. The BIT\* trajectory passes very close to obstacles at multiple locations, but MPC plans a smooth and dynamically feasible trajectory to avoid them.

Fig. 9 (b) has one unknown obstacle, through which the BIT\* trajectory is passing, as the obstacle was not present while generating the reference trajectory. MPC plans a smooth trajectory around the new obstacle. The replanned trajectory does not trigger morphing as the available space is still large enough to accommodate the unfolded configuration.

## B. Experiments

We validate the performance of the proposed framework in the experiments using MorphoCopter hardware. In addition to the design shown in [2], a 2D LiDAR sensor has been added to facilitate onboard obstacle sensing for deriving the obstacle avoidance cost function. We conduct experiments in 2 different environments. The first environment is an extremely narrow passage, and the other is a cluttered space with obstacles and tight spaces among them.

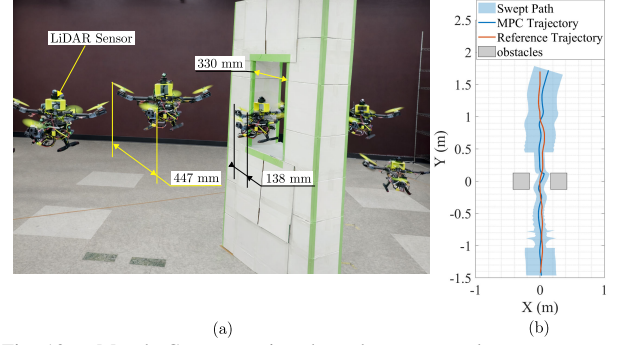


Fig. 10. MorphoCopter passing through an extremely narrow gap using online MPC trajectory planning with LiDAR perception (a) experiment video sequence (b) trajectories and swept path

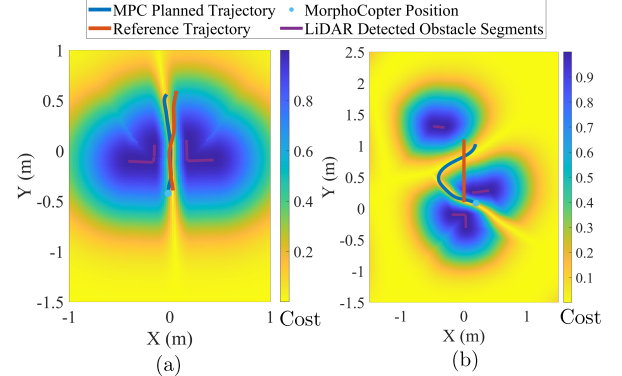


Fig. 11. Obstacle avoidance cost function for one of the iterations in experiments (a) extremely narrow gap environment (b) cluttered obstacles environment

1) *Extremely Narrow Gap*: In this experiment, a narrow passage of the width  $0.33\text{ m}$  has been constructed as shown in Fig. 10 (a). The figure shows the MorphoCopter safely passing through this challenging obstacle using the proposed MPC framework. Fig. 10 (b) shows the reference trajectory, MPC trajectory, and the swept path. The swept path illustrates that the MorphoCopter folds before entering the narrow gap at a safe distance and unfolds just after safely passing through it. Fig. 11 (a) shows the obstacle avoidance cost function for one of the iterations. In this figure, we can see that the line segments and cost are generated properly even with noisy measurements with the actual hardware, indicating the robustness of the framework.

2) *Cluttered Obstacles*: In this experimental setup, 3 cluttered obstacles, as shown in Fig. 12 (a), are arranged with narrow gaps among them. The narrowest passage is of the  $0.38\text{ m}$  width, and it is at around  $45^\circ$  angle with respect to the reference trajectory. In order to demonstrate



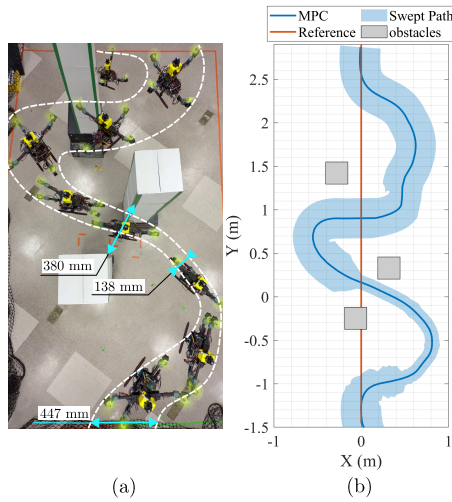


Fig. 12. MorphoCopter passing through a cluttered obstacles environment using online MPC trajectory planning with LiDAR perception (a) experiment video sequence (b) trajectories and swept path

the capability of the proposed MPC framework, the straight line reference connecting the start and goal is used, and the MPC is fully responsible for planning the trajectory and folding to avoid the obstacles. The MorphoCopter also has to adjust yaw and joint angle simultaneously. Fig. 11 (b) shows the snapshot of the MPC planning for one of the iterations along with the cost function. We can clearly see that the MPC trajectory has been adjusted by a large amount in the X direction to avoid the obstacles. Fig. 12 (b) shows the comparison of the reference trajectory with the MPC trajectory and the swept path. In the X direction, the MPC trajectory is deviating by around  $0.75\text{ m}$  from the reference trajectory at one point to avoid the obstacles, demonstrating the obstacle cost function safely guiding the MorphoCopter among the obstacles.

## V. CONCLUSIONS

This article introduces a novel general-purpose obstacle avoidance cost function that has a low cost in the ultra-narrow passages and uses only onboard 2D LiDAR data. The cost function has been implemented in the real-time application of planning the trajectory and morphing for the MorphoCopter. The simulations and experiments evaluated the performance of the framework and compared it with standard APF-like cost functions. The limitations of the current methodology include replanning only 2D space and the inability to predict the motion of the moving obstacles. We plan to address these issues in the future to make the methodology more robust.

## REFERENCES

- [1] D. Falanga, K. Kleber, S. Mintchev, D. Floreano and D. Scaramuzza, "The Foldable Drone: A Morphing Quadrotor That Can Squeeze and Fly," in *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 209–216, April 2019, doi: 10.1109/LRA.2018.2885575
- [2] H. Modi, H. Su, X. Liang, and M. Zheng, "MorphoCopter: Design, Modeling, and Control of a New Transformable Quad-Bi Copter," in *IEEE/ASME Transactions on Mechatronics*, doi: 10.1109/TMECH.2025.3643609
- [3] C. -X. Li, H. -N. Wu and T. Yang, "Coordinated Control of Flight and Morphing for Morphing Quadrotor via Reinforcement Learning," in *IEEE Transactions on Aerospace and Electronic Systems*, vol. 61, no. 5, pp. 12755–12766, Oct. 2025, doi: 10.1109/TAES.2025.3574295.
- [4] Y. Xie, M. Lu, R. Peng, and P. Lu, "Learning agile flights through narrow gaps with varying angles using onboard sensing," *IEEE Robot. Autom. Lett.*, vol. 8, no. 9, pp. 5424–5431, 2023.
- [5] J. Lin, L. Wang, F. Gao, S. Shen, and F. Zhang, "Flying through a narrow gap using neural network: An end-to-end planning and control approach," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2019, pp. 3526–3533, doi: 10.1109/IROS40897.2019.8967944.
- [6] D. Falanga, E. Mueggler, M. Faessler, and D. Scaramuzza, "Aggressive quadrotor flight through narrow gaps with onboard sensing and computing using active vision," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 5774–5781, doi: 10.1109/ICRA.2017.7989679.
- [7] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3529–3536, 2019.
- [8] M. Ryll, J. Ware, J. Carter, and N. Roy, "Efficient trajectory planning for high speed flight in unknown environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 732–738.
- [9] F. Gao and S. Shen, "Online quadrotor trajectory generation and autonomous navigation on point clouds," in *Proc. IEEE Int. Symp. Safety, Security, and Rescue Robot. (SSRR)*, 2016, pp. 139–146.
- [10] Z. Liu and L. Cai, "Simultaneous planning and execution for quadrotors flying through a narrow gap under disturbance," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 6, pp. 2644–2659, 2023, doi: 10.1109/TCST.2023.3283446.
- [11] A. Aikebaier, Q. Wang, Y. Bai, and Q. Wang, "Motion configuration planning method of morphing quadrotor," in *Proc. IEEE Int. Conf. Unmanned Syst. (ICUS)*, 2024, pp. 1597–1602, doi: 10.1109/ICUS61736.2024.10839796.
- [12] G. Cui, R. Xia, X. Jin, and Y. Tang, "Motion planning and control of a morphing quadrotor in restricted scenarios," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5759–5766, Jun. 2024, doi: 10.1109/LRA.2024.3396109.
- [13] N. Zhao, Y. Luo, C. Qin, X. Luo, R. Chen, and Y. Shen, "Attitude control for morphing quadrotor through model predictive control with constraints," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 10489–10495, doi: 10.1109/ICRA57147.2024.10610512.
- [14] X. Zhang, J. Ma, Z. Cheng, S. Huang, S. S. Ge, and T. H. Lee, "Trajectory generation by chance-constrained nonlinear MPC with probabilistic prediction," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3616–3629, 2021, doi: 10.1109/TCYB.2020.3032711.
- [15] C. Qin, N. Zhao, Q. Wang, Y. Luo, and Y. Shen, "Minimum snap trajectory planning and augmented MPC for morphing quadrotor navigation in confined spaces," *Drones*, vol. 9, no. 4, Art. no. 304, 2025, doi: 10.3390/drones9040304.
- [16] Y. Wu, Z. Han, X. Wu, Y. Zhou, J. Wang, Z. Fang, and F. Gao, "Shape-adaptive planning and control for a deformable quadrotor," *arXiv preprint arXiv:2505.15010*, 2025.
- [17] A. Papadimitriou, S. S. Mansouri, C. Kanellakis, and G. Nikolakopoulos, "Geometry aware NMPC scheme for morphing quadrotor navigation in restricted entrances," in *Proc. European Control Conf. (ECC)*, 2021, pp. 1597–1603, doi: 10.23919/ECC54610.2021.9655205.
- [18] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [19] Y. Koren and J. Borenstein, "Potential field methods and their inherent limitations for mobile robot navigation," in *Proceedings. 1991 IEEE International Conference on Robotics and Automation*, Sacramento, CA, USA, 1991, pp. 1398–1404, doi: 10.1109/ROBOT.1991.131754
- [20] R. B. Tilove, "Local obstacle avoidance for mobile robots based on the method of artificial potentials," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 1990, pp. 566–571.
- [21] D. Wang, L. Mu, B. Wang, Q. Li, and X. Xue, "UAV obstacle avoidance algorithm based on model predictive control and control barrier functions," *IFAC-PapersOnLine*, vol. 59, no. 20, pp. 405–410, 2025.
- [22] A. M. Ali, H. A. Hashim, and C. Shen, "MPC-based linear equivalence with control barrier functions for VTOL-UAVs," in *Proc. American Control Conf. (ACC)*, 2024.
- [23] R. Verschueren, M. Zanon, R. Quirynen, and M. Diehl, "acados: A modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, pp. 147–183, 2022.
- [24] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Batch informed trees (BIT\*): Informed asymptotically optimal anytime search," *Int. J. Robot. Res.*, vol. 39, no. 5, pp. 543–567, 2020.